

CK2 AI Discord Bot - Dev Log

Project Goal

Build a Discord AI assistant knowledgeable about Crusader Kings II. Use RAG with the CK2 Wiki as the knowledge source, ChromaDB as the vector database, and eventually connect retrieval to an LLM/Discord bot.

1. Environment & Wiki Access

- Set up Python project in PyCharm with `.venv`.
- Installed `requests` initially.
- Direct Python requests to the Paradox Wiki were intercepted by its security/challenge page.
- **Tried:** REST/API access through normal `requests`.
- **Abandoned:** Direct `requests` approach because of the security challenge.
- Installed Playwright and Chromium.
- **Successful approach:** Use Playwright to access the MediaWiki API through a real browser.

2. Wiki Page Discovery

- Switched project knowledge source from CK3 → CK2 Wiki.
- Used MediaWiki Action API `allpages` to discover pages.
- Implemented pagination using `continue` / `apcontinue`.
- Found 1,777 total namespace-0 pages.
- Discovered MediaWiki can distinguish redirects using `apfilterredirect`.
- Results:
 - 640 content/non-redirect pages
 - 1,137 redirect pages
- Created reusable `get_pages(redirect_filter, page)` function.
- **Decision:** Process the 640 content pages now; retain knowledge of redirects for possible aliases later.

3. Raw Wiki Scraper

- Used page IDs rather than titles for API requests.
- Created `download_page(single_page, page)`.
- Each page is saved as JSON containing:
 - `pageid`

- title
- wikitext

data/content/raw/

- **Successfully downloaded** all 640 content pages.
- Mod pages are intentionally being kept for now. They can be classified/filterable later instead of throwing away data during scraping.

4. Wikitext Investigation

Inspected the raw Warfare article (`pageid 40`) against the actual webpage.

Found:

- `''' '''` → bold
- `[[ ]]` → wiki links
- `==, ===, etc.` → headings
- `*` → bullet points
- `;` → definition/indent-style formatting
- `{{ }}` → many different templates: notes, keyboard keys, images, etc.
- Tables contain useful information but are complicated in raw wikitext.
- Table of contents is generated rather than stored directly in the source.

Conclusion: Writing our own wikitext parser would be unnecessarily complicated and risks deleting useful information.

5. MediaWiki Parsing Experiment

- Tested MediaWiki `action=parse`.
- Successful: MediaWiki converts the raw article into structured HTML itself.
- Templates are expanded.
- Tables are properly represented.
- Headings/lists/paragraphs survive.
- Images have HTML representations/placeholders.
- Generated elements such as the table of contents also appear.

Tried: Sending our locally archived wikitext back through `action=parse` using POST.

Abandoned: POST requests trigger the wiki's security page.

Moving forward with: `action=parse&pageid=...` through Playwright.

Raw JSON remains untouched as our archive/source data.

6. HTML Processing

- Added `BeautifulSoup` to parse MediaWiki `action=parse` HTML.
- Confirmed headings, paragraphs, lists, and tables can all be extracted cleanly.
- Removed generated TOC with `div#toc`.
- Removed `[edit | edit source]` spans using `.mw-editsection`.
- Decided to preserve page structure instead of flattening everything into one string.
- Added section tracking with:
 - `heading`
 - `level`
 - `content`
- Added synthetic `Introduction` section for content before first heading.
- Paragraphs stored as typed content objects.
- Added recursive list processing:
 - preserves nested `<ul>` / `<ol>`
 - avoids duplicate nested-list extraction
 - stores child items under `children`
- Preserved ordered vs unordered list type.
- Added table processing using `<tr>`, `<th>`, and `<td>`.
- Warfare page now converts successfully into structured processed JSON and matches the rendered wiki page closely.
- Next: save processed output to `data/content/processed/40.json`, then generalize processor to all content pages.